



PGP Commands - Usage Guide & Examples

Overview

Prerequisites

1. Key Management (RepoPgpKey)

Generate a Key Pair

Import an External Public Key

Import an External Private Key

List Keys

Update a Key

Delete a Key

2. Encrypting Files (PGPEncrypt)

Encrypt from base64

Encrypt from plain text

3. Decrypting Files (PGPDecrypt)

Decrypt from base64

Decrypt ASCII-armored ciphertext

4. Public Key Distribution (PGPKeyLink)

5. Command Chain Examples

Pre-processor: Decrypt inbound file before batch import

Post-processor: Encrypt outbound export and upload via SFTP

Full roundtrip test

6. Frontend: PGP Key Management

Features

Typical Workflow

7. Security Notes

8. Database Model

9. API Reference Summary

Overview

SISos now includes native PGP encryption/decryption commands and a key management system. This enables encrypted file exchange with external partners (banks, reinsurers) directly within command chains and batch workflows.

Commands available:

- `RepoPgpKey` — CRUD + key pair generation
- `PGPEncrypt` — Encrypt content with a public key
- `PGPDecrypt` — Decrypt content with a private key
- `PGPKeyLink` — Generate a public download URL for a public key

Frontend: Settings → PGP Keys (admin only)

Feature flag: `Security.enablePgpKeyDistribution` in ConfigProfile must be `true` for the public key download endpoint to work.

Prerequisites

1. Run the database migration to create the `PgpKey` table
2. Enable the feature flag in ConfigProfile if you need key distribution:

```
{
  "Security": {
    "enablePgpKeyDistribution": true
  }
}
```

1. Key Management (RepoPgpKey)

Generate a Key Pair

Generates a linked public + private key pair. The private key is encrypted at rest using AES-256.

```
// Generate with auto-generated passphrase
exe("RepoPgpKey", {
```

```

    operation: "GENERATE",
    entity: {
      name: "BankX Exchange Key",
      category: "banking"
    }
  })

// Generate with custom passphrase
exe("RepoPgpKey", {
  operation: "GENERATE",
  entity: {
    name: "ReinsurerY Key",
    category: "reinsurance"
  },
  generatePassphrase: "MySecurePassphrase123"
})

```

Response: Returns an array with 2 records — the public key (full) and the private key (material stripped for security).

Import an External Public Key

When a client/partner provides their public key for encrypting outbound files:

```

exe("RepoPgpKey", {
  operation: "ADD",
  entity: {
    name: "ClientZ Public Key",
    category: "banking",
    keyType: "PUBLIC",
    armoredKey: "-----BEGIN PGP PUBLIC KEY BLOCK-----\nmQINB
G...\n-----END PGP PUBLIC KEY BLOCK-----"
  }
})

```

Import an External Private Key

For decrypting files from a partner that encrypted with a specific key pair:

```
exe("RepoPgpKey", {
  operation: "ADD",
  entity: {
    name: "Legacy Decryption Key",
    category: "migration",
    keyType: "PRIVATE",
    armoredKey: "-----BEGIN PGP PRIVATE KEY BLOCK-----\nmQINB
G...\n-----END PGP PRIVATE KEY BLOCK-----",
    passphrase: "keyPassphrase"
  }
})
```

The private key and passphrase are automatically encrypted before storing.

List Keys

```
// All keys
exe("RepoPgpKey", { operation: "GET" })

// Filter by type
exe("RepoPgpKey", { operation: "GET", filter: "keyType='PUBLIC'" })

// Filter by category
exe("RepoPgpKey", { operation: "GET", filter: "category='banking'" })

// Paginated
exe("RepoPgpKey", { operation: "GET", size: 10, page: 0 })
```

Private key material is automatically stripped from GET responses for security.

Update a Key

Only name, category, active status, and expiry can be updated. Key material is immutable.

```
exe("RepoPgpKey", {
  operation: "UPDATE",
  entity: {
    id: 5,
    name: "BankX Exchange Key (Rotated)",
    category: "banking",
    active: true,
    expiresAt: "2027-12-31T00:00:00Z"
  }
})
```

Delete a Key

```
exe("RepoPgpKey", { operation: "DELETE", entity: { id: 5 } })
```

2. Encrypting Files (PGPEncrypt)

Encrypts content using a stored public key. Use this for outbound files to external partners.

Encrypt from base64

```
// Encrypt a file that was fetched as base64
exe("PGPEncrypt", {
  keyId: 3, // ID of the PUBLIC key to encrypt with
  base64: "SGVsbG8gV29ybGQ=", // File content as base64
  outputFileName: "report.pgp", // Optional output filename
  armor: true // ASCII-armored output (default true)
})
```

Response:

```
{
  "ok": true,
  "msg": "File encrypted successfully",
  "outData": {
    "base64": "LS0tLS1CRUdJTi...",
    "fileName": "report.pgp"
  }
}
```

Encrypt from plain text

```
exe("PGPEncrypt", {
  keyId: 3,
  plainText: "Sensitive CSV data here...\nrow1,value1\nrow2,value2",
  armor: true
})
```

3. Decrypting Files (PGPDecrypt)

Decrypts content using a stored private key. Use this for inbound encrypted files.

Decrypt from base64

```
exe("PGPDecrypt", {
  keyId: 4, // ID of the PRIVATE key
  base64: "LS0tLS1CRUdJTi...", // Encrypted file as base64
  outputFileName: "payroll.csv",
  decodeString: true // Also return UTF-8 decoded text
})
```

Response:

```
{
  "ok": true,
  "msg": "File decrypted successfully",
  "outData": {
    "base64": "SGVsbG8gV29ybGQ=",
    "fileName": "payroll.csv",
    "utf8": "Hello World"
  }
}
```

Decrypt ASCII-armored ciphertext

```
exe("PGPDecrypt", {
  keyId: 4,
  cipherText: "-----BEGIN PGP MESSAGE-----\n...",
  decodeString: true
})
```

4. Public Key Distribution (PGPKeyLink)

Generates a signed, public URL where external partners can download your public key as an `.asc` file.

```
exe("PGPKeyLink", { keyId: 3 })
```

Response:

```
{
  "ok": true,
  "msg": "Public key link generated",
  "outData": {
    "url": "https://api.sisos.com/int/keys/M3xhYmNkZWY...",
    "keyId": 3,
    "fingerprint": "A1B2C3D4E5F6..."
  }
}
```

```
}  
}
```

The URL can be shared with external partners. Accessing it via browser or `curl` downloads the `.asc` public key file.

Requirement: The `Security.enablePgpKeyDistribution` flag must be `true` in `ConfigProfile`, otherwise the endpoint returns 404.

5. Command Chain Examples

Pre-processor: Decrypt inbound file before batch import

Configure a Chain with the following formula code, then set the chain ID as the `preProcessorId` on the corresponding `ImportConfig` record:

```
// Pre-processor chain formula  
var decrypted = doCmd("PGPDecrypt", {  
    base64: _base64,  
    keyId: 4,  
    decodeString: true  
});  
_base64 = decrypted.outData.base64;  
return _context;
```

Post-processor: Encrypt outbound export and upload via SFTP

```
// Chain formula: encrypt then upload  
var encrypted = doCmd("PGPEncrypt", {  
    base64: _base64,  
    keyId: 3,  
    armor: true  
});  
  
var uploaded = doCmd("Util.UploadBytes", {  
    bytes: encrypted.outData.base64,
```



```

        fileName: "export_" + _date + ".pgp"
    });

    var sftp = doCmd("Util.DoSftpUpload", {
        host: "sftp.clientz.com",
        username: "sisos",
        remotePath: "/incoming/export_" + _date + ".pgp",
        base64: encrypted.outData.base64
    });

    return _context;

```

Full roundtrip test

```

// 1. Generate key pair
var keys = doCmd("RepoPgpKey", {
    operation: "GENERATE",
    entity: { name: "Test Key", category: "test" }
});
var publicKeyId = keys.outData[0].id;
var privateKeyId = keys.outData[1].id;

// 2. Encrypt
var encrypted = doCmd("PGPEncrypt", {
    keyId: publicKeyId,
    plainText: "Hello, this is a secret message!"
});

// 3. Decrypt
var decrypted = doCmd("PGPDecrypt", {
    keyId: privateKeyId,
    base64: encrypted.outData.base64,
    decodeString: true
});

```

```
// decrypted.outData.utf8 === "Hello, this is a secret message!"
```

6. Frontend: PGP Key Management

Access via **Settings** → **PGP Keys** (admin only).

Features

- **Key list** with filters by name, category, and type (Public/Private)
- **Import Key** — paste an ASCII-armored public or private key
- **Generate Key Pair** — creates a linked public + private key pair with optional passphrase
- **Copy Public Link** — generates and copies a signed download URL to clipboard (for public keys)
- **Edit** — update name, category, active status, expiration date
- **Delete** — permanently remove a key

Typical Workflow

1. Go to Settings → PGP Keys
2. Click **Generate Key Pair**, enter a name (e.g., "BankX Exchange") and category
3. On the public key row, click the link icon to copy the public download URL
4. Share the URL with the external partner
5. Import the partner's public key via **Import Key**
6. Configure command chains to use the appropriate key IDs for encryption/decryption

7. Security Notes

- **Private keys** are encrypted at rest using AES-256 with the application secret
- **Passphrases** are also encrypted before storage

- Private key material is **never returned** in GET/list responses
- The public key download endpoint is **disabled by default** — must be enabled per environment via `Security.enablePgpKeyDistribution`
- Download URLs are **HMAC-signed** to prevent enumeration
- The `Version` header comment is stripped from generated keys to avoid exposing library metadata

8. Database Model

Table: `PgpKey`

Column	Type	Description
id	int (PK)	Auto-increment ID
name	string	User-friendly name
category	string	Grouping (e.g., "banking")
keyType	string	"PUBLIC" or "PRIVATE"
fingerprint	string	PGP fingerprint
armoredKey	string	ASCII-armored key (encrypted for private)
passphrase	string	Encrypted passphrase (private keys only)
expiresAt	datetime?	Optional expiration
active	bool	Active/inactive toggle
createdBy	string	User who created the key
created	datetime	Creation timestamp

9. API Reference Summary

Command	Input	Output
<code>RepoPgpKey</code>	operation, entity, filter, size, page, generatePassphrase	List of PgpKey records
<code>PGPEncrypt</code>	keyId (public), base64 or plainText, outputFileName, armor	base64, fileName

Command	Input	Output
<code>PGPDecrypt</code>	keyId (private), base64 or cipherText, outputFileName, decodeString	base64, fileName, utf8
<code>PGPKeyLink</code>	keyId (public)	url, keyId, fingerprint

Public endpoint: `GET /int/keys/{token}` — Downloads public key as `.asc` file
(requires `enablePgpKeyDistribution` flag)